Lecture 3: Lists

CSI2110 – Data Structures and Algorithms

Lecture study notes

Source document: `../lec03-list.pdf`. These notes synthesize the source and add worked explanations. References use PDF page numbers. Pseudocode includes boundary cases.

Contents

1 Index, position, and sequence **2**

2 Array-list ADT **2**

3 Extendable arrays and amortized cost **2**

3.1 Fixed-increment growth 2

3.2 Doubling growth 3

4 Positional-list ADT and linked implementation **3**

5 Sequence ADT and implementation tradeoffs **4**

6 Move-to-front encoding **4**

7 Study checks **5**

1 Index, position, and sequence

Source: PDF pages 3–7,17–19,25. A list is a collection in linear order. An array-list ADT accesses entries by their zero-based index (rank). A positional-list ADT accesses an entry through a position: a handle to the entry, usually a node reference. A sequence ADT supports both and adds bridge methods.

In $[A, B, C, D]$, C has index 2. Inserting X before C yields $[A, B, X, C, D]$. The index of C becomes 3, but a position p identifying C continues to identify the same entry.

2 Array-list ADT

Source: PDF pages 7–11. For size n, `get(i)`, `set(i,e)`, and `remove(i)` require $0 \le i < n$. Insertion `add(i,e)` permits $0 \le i \le n$, including appending at n. Replacement returns the old element; removal returns the deleted element. `size()` and `isEmpty()` are support operations.

From $[4, 7, 9]$, `set(1,5)` gives $[4, 5, 9]$ and returns 7. Separately, `add(1,5)` gives $[4, 5, 7, 9]$ and increases size.

Represent entries in $V[0..n-1]$ of a capacity-N array; maintain $0 \le n \le N$.

```
add(i, e):
    validate 0 <= i <= n
    ensure capacity for n + 1 entries
    for j = n - 1 down to i:
        V[j + 1] = V[j]
    V[i] = e
    n = n + 1

remove(i):
    validate 0 <= i < n
    old = V[i]
    for j = i to n - 2:
        V[j] = V[j + 1]
    n = n - 1
    V[n] = null
    return old
```

Insertion shifts $n - i$ entries, starting at the right to avoid overwriting values. Removal shifts $n - i - 1$ entries. Both take $O(n)$ worst-case time. Access, replacement, size, and emptiness are $O(1)$. Without resizing, append and last removal are $O(1)$. Allocated space is $O(N)$.

3 Extendable arrays and amortized cost

Source: PDF pages 13–15. When full, allocate a larger array, copy existing entries, and append. Let $T(m)$ be the total time of m appends from an empty list of capacity one. Amortized time is $T(m)/m$.

3.1 Fixed-increment growth

For new capacity $N + c$, where $c > 0$ is fixed, copies form an arithmetic progression. With $c = 2$, capacities are $1, 3, 5, 7, \ldots$ and k resizes copy

$$1 + 3 + \cdots + (2k - 1) = k^2.$$

Since $k = \Theta(m)$ for fixed c, $T(m) = \Theta(m^2)$ and amortized cost is $\Theta(m)$ per append.

3.2 Doubling growth

For new capacity $2N$, copied counts are $1, 2, 4, 8, \ldots$. Across m appends their sum is less than $2m$:

$$1 + 2 + \cdots + 2^k = 2^{k+1} - 1 < 2m.$$

Adding the m writes gives $T(m) = \Theta(m)$ and amortized $\Theta(1)$ append.

Growth	Total for m appends	Amortized append
Fixed increment	$\Theta(m^2)$	$\Theta(m)$
Doubling	$\Theta(m)$	$\Theta(1)$

Key idea. One doubling append can still cost $\Theta(n)$ when it resizes. Amortized cost describes a sequence, not a worst-case individual operation and not a probabilistic average.

Middle insertion still shifts entries and remains $O(n)$. With doubling and no deletions, capacity is $O(n)$ for a nonempty list; without shrinking, many deletions can leave capacity much larger than size.

4 Positional-list ADT and linked implementation

Source: PDF pages 17–23. Accessors `first()`, `last()`, `before(p)`, and `after(p)` return positions. Updates `addFirst(e)`, `addLast(e)`, `addBefore(p,e)`, and `addAfter(p,e)` return the inserted position. `set(p,e)` returns the old element; `remove(p)` returns the removed element and invalidates the position. A position must belong to the list and still be valid. Sentinels are not user positions.

Use doubly linked nodes with element, previous, and next fields and header/trailer sentinels. All insertions share one helper:

```
addBetween(e, pred, succ):
    p = new Node(e, pred, succ)
    pred.next = p
    succ.prev = p
    size = size + 1
    return p

addAfter(p,e): return addBetween(e,p,p.next)
addBefore(p,e): return addBetween(e,p.prev,p)
addFirst(e): return addBetween(e,header,header.next)
addLast(e): return addBetween(e,trailer.prev,trailer)
```

```
remove(p):
    validate p
    old = p.element
    p.prev.next = p.next
    p.next.prev = p.prev
    size = size - 1
    invalidate p
    return old
```

These operations all take $O(1)$ given a valid position. Searching for a position can add $O(n)$ time. Space is $O(n)$. A boundary accessor returns no user position if there is no neighbor.

5 Sequence ADT and implementation tradeoffs

Source: PDF pages 25,28–35. A sequence combines indexed and positional operations, plus `atIndex(i)` to obtain the position of rank i and `indexOf(p)` to obtain the rank of p.

An array sequence stores position objects with an element and maintained rank. A circular array maps logical rank i to $(f + i) \bmod N$. An update can require both moving entries and updating ranks. A doubly linked sequence supports local position updates quickly, but rank translation requires traversal.

Operation	Array sequence (slide design)	Linked sequence
`size()`, `isEmpty()`	$O(1)$	$O(1)$
`get(i)`, `set(i,e)`	$O(1)$	$O(n)$
`add(i,e)`, `remove(i)`	$O(n)$	$O(n)$
`addFirst(e)`	$O(n)$	$O(1)$
`addLast(e)`	$O(1)$ if room	$O(1)$
`addBefore(p,e)`, `addAfter(p,e)`	$O(n)$	$O(1)$
`set(p,e)`	$O(1)$	$O(1)$
`remove(p)`	$O(n)$ worst	$O(1)$
`atIndex(i)`, `indexOf(p)`	$O(1)$	$O(n)$

In the array design, front insertion is linear because all stored logical ranks must be updated, even if circular storage avoids moving entries. These are costs of the lecture's design; other representations can change the tradeoffs. Resizing makes an individual array append $O(n)$.

6 Move-to-front encoding

Source: PDF pages 26–27,36. Start with $S = [1, 2, \ldots, m]$. For each requested symbol x, output its current zero-based rank and move it to the front. Maintain `loc[x]`, a position reference, not a rank that can become stale.

```
MTF(x):
    p = loc[x]
    r = S.indexOf(p)
    S.remove(p)
    loc[x] = S.addFirst(x)
    return r
```

Removing p invalidates it; store the newly returned position.

Request	Before	Rank	After
3	$[1, 2, 3, 4, 5]$	2	$[3, 1, 2, 4, 5]$
4	$[3, 1, 2, 4, 5]$	3	$[4, 3, 1, 2, 5]$
3	$[4, 3, 1, 2, 5]$	1	$[3, 4, 1, 2, 5]$
1	$[3, 4, 1, 2, 5]$	2	$[1, 3, 4, 2, 5]$

Thus $Y = [3, 4, 3, 1]$ encodes as $[2, 3, 1, 2]$.

In an array sequence, rank lookup is $O(1)$ but remove and front insertion are $O(m)$. In a linked sequence, rank lookup is $O(m)$ but both updates are $O(1)$. Both therefore take $O(m)$ worst case per request. For n requests plus initialization, total time is $O(m + nm)$ and working space is $O(m)$, excluding output.

7 Study checks

1. Why is `add(n,e)` legal but `get(n)` illegal? Insertion can use the slot after the last entry; access requires an existing entry.
2. Why does a position survive insertion before it? It identifies an entry rather than its rank.
3. Does amortized constant-time append imply constant-time middle insertion? No: shifting remains linear.
4. What dominates move-to-front? Rank maintenance in the array design; rank lookup in the linked design.